

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



EMBEDDED SYSTEMS (CO3053)

ASSIGNMENT REPORT

DESIGN OF A GREENHOUSE MICRO-CLIMATE MONITORING SYSTEM

Instructor: Phạm Hoàng Anh

Members: Nguyễn Quốc Thắng – 2353122

Võ Hoàng Nguyên – 2352844

Ngô Nguyễn Thành Nhân – 2352849

Contents

1	Problem and Design Approach	3
1.1	Problem statement	3
1.2	Six families of design challenges and their priority	3
1.3	Shared architecture	4
1.3.1	Four layers, hybrid processing, local gateway	4
1.3.2	Sensor placement	5
1.3.3	Node hardware	5
2	Criterion 1 – Communication Infrastructure	6
2.1	What the infrastructure must provide	6
2.2	Choosing the topology	6
2.3	Design A: wired RS-485 ring bus with Modbus RTU	7
2.3.1	Ring bus: surviving a cable break	7
2.3.2	Protocol: synchronised snapshot, then polling	7
2.3.3	Power and protection	8
2.4	Design B: wireless LoRa star with TDMA	8
2.4.1	Radio link budget	8
2.4.2	Channel access: TDMA instead of random access	9
2.4.3	Reliable delivery	9
2.4.4	Adaptive reporting: saving energy without delaying alerts	9
2.4.5	Energy	10
2.4.6	Security	10
2.5	Summary of the two infrastructures	10
3	Criterion 2 – Data Correctness	11
3.1	Where errors come from	11
3.2	Sensor layer: prevent errors at the source	11
3.3	Node layer: clean every sample	12
3.4	Gateway layer: catch what a single node cannot see	13
3.4.1	Detecting faulty sensors	13
3.4.2	Zone value: tolerate one bad node per zone	13
3.4.3	Real event or faulty sensor?	14

3.4.4	Missing data vs. late data	14
3.5	Error scenarios at a glance	14
4	Criterion 3 – Data Analysis for Alerts	16
4.1	The two ways an alert system fails	16
4.2	Processing pipeline	16
4.3	The alert state machine	17
4.4	Four mechanisms against false alarms	19
4.4.1	Persistence over a sliding window	19
4.4.2	Hysteresis – avoiding a “Zeno” alarm	19
4.4.3	Zone median	20
4.4.4	Late data is not missing data (watermark)	20
4.5	Keeping real alarms fast	20
4.6	Alerts from derived indices and from space	20
4.7	Device alerts are separate	21
5	Performance Evaluation	22
5.1	The 10 steps in brief	22
5.2	Metrics chosen for each design	22
5.3	Key results	23
5.3.1	Does the network change the alert delay? (paired comparison)	23
5.3.2	Does adaptive reporting save energy? (criterion 1)	24
5.4	Evaluation changed the design	25
6	Comparison and Recommendation	26
7	Conclusion	28
A	Simulator	30

Chapter 1

Problem and Design Approach

1.1 Problem statement

The task is to design a system that monitors air temperature, air humidity and light intensity inside a 20×50 m greenhouse, and to discuss three criteria:

1. **Communication infrastructure** – how measurements get from the sensors to a processing point (Chapter 2).
2. **Data correctness** – how we make sure the values we act on are right (Chapter 3).
3. **Data analysis for alerts** – how we decide when to warn the grower (Chapter 4).

At least two designs must be proposed and compared. We propose **Design A** (wired RS-485 bus) and **Design B** (wireless LoRa star). Both share the same sensing and the same alert software; they differ only in the communication layer, so the comparison is fair.

Two properties of the problem drive every decision:

- **The climate is not uniform.** Areas near sunny walls, fans or misters differ from the centre, so one sensor cannot represent $1,000 \text{ m}^2$. We need many measuring points and a way to combine them.
- **The physics is slow, the timing is soft.** Greenhouse air changes over minutes, not milliseconds. A delay of a few seconds causes no harm, so this is a *soft* real-time system (Kopetz). What matters more is that alerts are *trustworthy*: no false alarms, no missed events.

1.2 Six families of design challenges and their priority

Marwedel groups embedded design challenges into six families. They cannot all be optimised at once, so we ranked them for each design. When two goals conflict, the higher-ranked one wins.

The key consequence: in B, energy and latency pull in opposite directions (sending rarely saves battery but delays data). Instead of picking one fixed compromise, B uses *adaptive reporting* (Section 2.4.4).

Table 1.1: Priority of the six challenge families (H = high, M = medium, L = low)

Challenge family	A	B	How it shows up here, and why A and B differ
1. Dependability	H (1st)	H (1st)	Drifting sensors, lost data, false alarms. Top for both: an alert system nobody trusts is useless.
2. Physical/cyber mismatch	H (2nd)	M (3rd)	A continuous temperature field is sampled at a few points; a hard threshold on noisy data flips on and off (a Zeno-like effect). B also delivers data with variable delay.
3. Resource efficiency	M (3rd)	H (2nd)	A is powered through the cable, so only cable cost matters. For B, an empty battery = a dead node, so energy is part of dependability .
4. Complexity	M	M–H	B must build its own channel access, time sync, buffering and encryption.
5. Societal impact	L	M	Alarm fatigue for growers (both); radio-band rules, battery waste and data security (B).
6. Data & machine learning	L	L	Data volume is small; rules must be explainable to the grower. ML is left for future work.

1.3 Shared architecture

1.3.1 Four layers, hybrid processing, local gateway

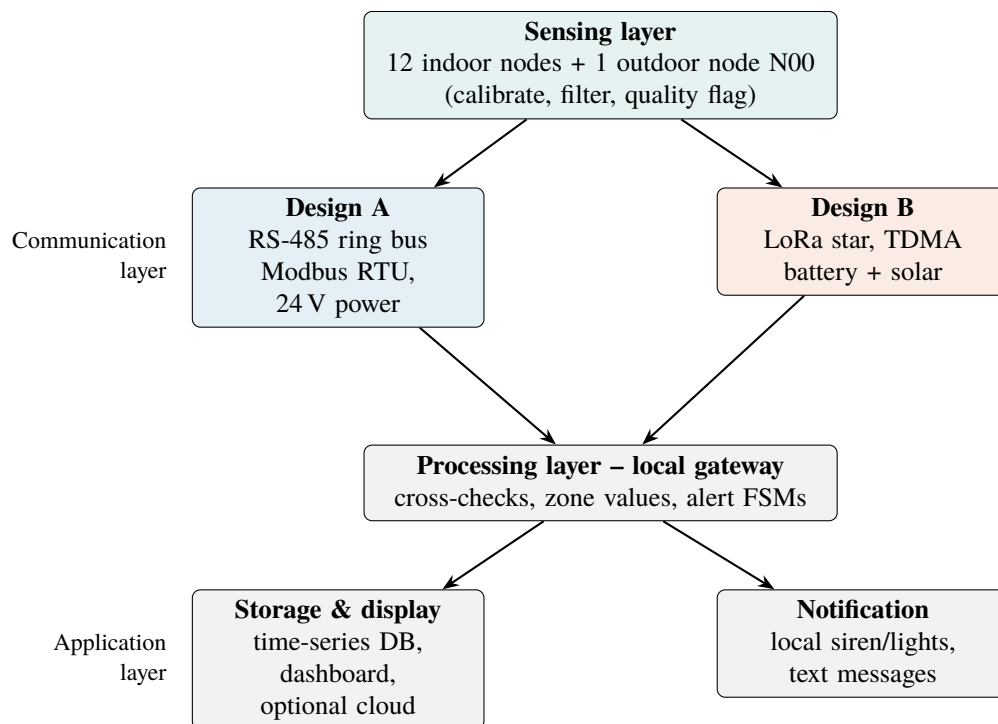


Figure 1.1: Shared four-layer architecture; A and B differ only in the communication layer

- **Hybrid processing.** Each function runs where it has enough information to be done correctly. *Nodes* do what needs raw samples: calibration, median filtering, range checks, quality flags – so they send one clean value instead of five raw ones. The *gateway* does what needs many nodes at once: detecting a drifting sensor by comparing neighbours, computing zone values, running the alert logic.
- **Local gateway, cloud optional.** All alert logic runs on a gateway inside the greenhouse, so the

local siren still works when the Internet is down.

1.3.2 Sensor placement

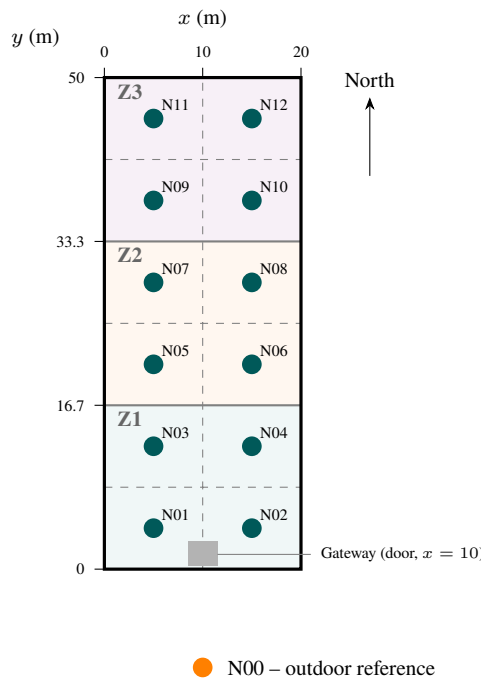


Figure 1.2: 12 nodes on a 2×6 grid, three zones Z1–Z3 of 4 nodes each, outdoor node N00 and the gateway at the door

Where the numbers come from. We use a 2×6 grid (12 nodes); each node sits at the centre of a 10×8.3 m cell, so no point of the greenhouse is more than $\sqrt{5^2 + 4.2^2} \approx 6.5$ m from a node. The long axis is split into three 20×16.7 m zones of exactly 4 nodes each. Four nodes per zone is the minimum that keeps a trustworthy zone median (≥ 3 good nodes, Section 3.4.2) when one node fails. Alternatives: 2×4 (8 nodes) gives only two large zones of 25 m; 2×8 (16 nodes) costs 33 % more nodes to reduce the worst distance by only 0.6 m. Nodes hang at crop-canopy height (default 1.5 m, adjustable) inside a ventilated radiation shield. N00, outside the south wall, tells weather changes apart from problems inside the greenhouse.

1.3.3 Node hardware

Both designs use **identical sensors**, so measurement error is not a difference between them: an SHT40 for temperature/humidity (factory-calibrated, has a built-in heater to clear condensation) and a TSL2591 for light (wide dynamic range, up to about 88 klux). Midday light under the roof can come close to that limit, so the node reduces the sensor's gain and integration time automatically, and any reading at full scale is flagged SUSPECT instead of being trusted. The microcontroller, radio and power supply follow each design's priorities (Chapter 2).

Chapter 2

Criterion 1 – Communication Infrastructure

2.1 What the infrastructure must provide

Whatever the technology, the communication layer must deliver four things to the gateway:

1. **Completeness** – every sample arrives eventually, even after a short outage (retry or backfill).
2. **Timeliness** – samples arrive fast enough for the alert logic (a few tens of seconds is fine; Chapter 1).
3. **Integrity and ordering** – corrupted, duplicated or forged packets are detected; every sample has a timestamp.
4. **No single point of failure in the network** – one broken cable or one dead node must not blind a whole area.

Design B has one extra requirement: **energy autonomy**, because its nodes run on batteries.

2.2 Choosing the topology

Table 2.1: Candidate network topologies

Topology	Strength	Weakness	Decision
Bus	Least cable; RS-485 is natively a multi-drop bus	A cable break cuts off every node behind it	Design A , upgraded to a <i>ring bus</i> to remove the weakness
Star	Nodes fail independently	Wired: 12 separate cable runs; wireless: every node must be in range	Design B : the farthest node is about 46 m from the gateway, well within one LoRa hop
Mesh	Redundant paths, long reach	Relay nodes must stay awake (drains batteries); routing; extra delay	Not needed at 50 m

2.3 Design A: wired RS-485 ring bus with Modbus RTU

2.3.1 Ring bus: surviving a cable break

A normal bus loses every node beyond a cable break. The cable runs up the west column of nodes, across the north end and down the east column, so **both ends come back to the gateway**, which has two RS-485 ports, P1 and P2 (Figure 2.1).

- **Normal mode:** P1 is the Modbus master and polls all nodes. P2 only listens. Because P2 hears every frame on the cable, it acts as a *continuity monitor*: if P2 stops hearing P1, the cable is broken somewhere.
- **Split mode (after a break):** P1 polls the nodes on its side, P2 polls the rest. **No node is lost.** The break lies between the last node answering P1 and the last node answering P2, so maintenance knows which cable segment to repair.

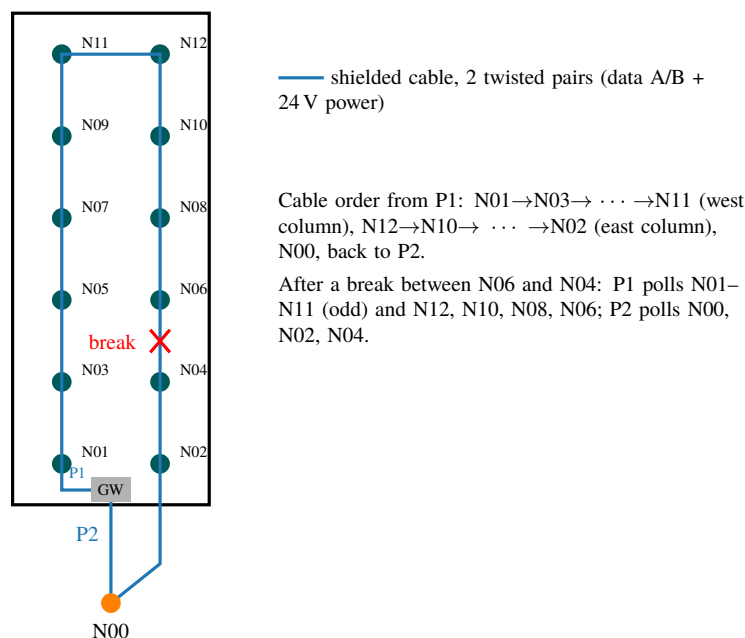


Figure 2.1: Design A: RS-485 cable up one column and down the other, with both ends returning to the gateway

Where the numbers come from. The cable is about 130 m: two columns of about 45 m each, 10 m across the north end, and the loop out to N00 and back to the door. This is far below the 1,200 m limit of RS-485, and 13 nodes plus 2 gateway transceivers stay well below the 32 standard loads a bus can drive.

2.3.2 Protocol: synchronised snapshot, then polling

Every 30 s the gateway:

1. broadcasts “sample now” with the current time – all nodes measure at the same instant (one coherent spatial snapshot) and their clocks are synchronised for free;
2. reads each node’s registers (value, quality flags, sequence number, status) with a Modbus read request;

3. retries once on a CRC error or timeout; if it still fails, marks that node MISSING for this cycle.

Where the numbers come from. *Why 30 s?* A failed fan raises temperature by roughly 0.3–0.5 °C per minute, so a 30 s period gives about ten samples while the temperature climbs a few degrees – enough for a reliable decision. *Why 19,200 bps?* At this speed one request/response takes about 26 ms, so polling all 13 nodes takes about a third of a second: the bus is idle about 99 % of the time, leaving room for retries and backfill. A lower speed than the maximum is also more tolerant to electrical noise from pumps and fans.

2.3.3 Power and protection

The second twisted pair carries 24 V; each node has its own step-down converter. 24 V is a safe extra-low voltage in a wet environment, and the voltage drop over 130 m is small because each node draws only a few milliamperes. Each node has surge (TVS) protection; the gateway ports are galvanically isolated; enclosures are IP65 with waterproof cable glands.

2.4 Design B: wireless LoRa star with TDMA

2.4.1 Radio link budget

Before choosing radio settings we check that the farthest node can reach the gateway with margin to spare (Table 2.2).

Table 2.2: Link budget to the farthest node (Design B)

Item	Value
Transmit power	+14 dBm
Antenna gain (transmitter + receiver)	+4 dBi
Free-space path loss (46 m)	−65.0 dB
Extra loss allowance: wet foliage, metal frame, people/carts	−35 dB
Estimated received power	−82.0 dBm
SX1276 sensitivity, SF7, 125 kHz bandwidth	−123 dBm
Link margin	about 40 dB

Where the numbers come from. **+14 dBm** is a standard output setting of the SX1276 radio (power at the chip, before the antenna; the legal check uses radiated power, see below). **+4 dBi**: a simple 2 dBi whip antenna on each side. **46 m**: the farthest nodes (N11, N12 at the north end) are $\sqrt{5^2 + 45.8^2} \approx 46$ m from the gateway at the south door. **65.0 dB** comes from the free-space formula $F SPL = 20 \log_{10} d_{km} + 20 \log_{10} f_{MHz} + 32.44$ with $d = 0.046$ km and $f = 923$ MHz. **35 dB** is a deliberately pessimistic *assumption* (not a measurement) for leaves full of water, the steel structure and moving people; it must be checked on site. **−82.0 dBm** is simply the sum of the lines above. **−123 dBm** is the datasheet sensitivity at the fastest LoRa setting (SF7). **About 40 dB** is the difference. A margin of about 20 dB is usually considered comfortable, so we have plenty.

Consequence. Because the margin is large even under pessimistic assumptions, we use **SF7**, the fastest spreading factor. A packet then takes only about 67 ms on air, which minimises both energy per packet and the chance of collisions. The gateway can also *lower* each node’s transmit power while keeping about 20 dB margin – too much margin is wasted battery.

Regulatory check. Legal limits apply to the power actually *radiated*, not to the power at the chip. The

node's radiated power is

$$\text{EIRP} = P_{TX} + G_{\text{antenna}} = 14 \text{ dBm} + 2 \text{ dBi} = 16 \text{ dBm} (\approx 40 \text{ mW}), \quad (2.1)$$

which equals about 13.9 dBm ERP ($\approx 24 \text{ mW}$; ERP is EIRP minus 2.15 dB). In Vietnam, LPWAN radios in the 920–923 MHz band fall under QCVN 122:2020/BTTTT; before that regulation, LoRa devices had to use the generic short-range-device allocation limited to 25 mW ERP. Our node stays within even this stricter limit, and in practice the power-control loop will run it lower. The same family of rules restricts channel occupancy by a *duty-cycle* limit (commonly 1 %) or by *listen-before-talk*. Our worst case – dense mode, one 67 ms packet every 30 s – is a duty cycle of 0.22 %. If listen-before-talk is required, the SX1276's channel-activity detection can check the channel at the start of each slot; this takes a few milliseconds and fits easily in the 1.5 s slot. The exact limits must be confirmed against the current text of the regulation before deployment.

2.4.2 Channel access: TDMA instead of random access

If 13 nodes transmit at random times (pure ALOHA), two packets sometimes overlap and both are lost. With 13 nodes sending a 67 ms packet every 60 s, the standard ALOHA formula $P_{\text{collision}} = 1 - e^{-2G}$ (where G is the fraction of time the channel is busy) gives about 3 % lost packets – each one costing a retransmission and battery. We therefore give each node its own time slot (Figure 2.2).

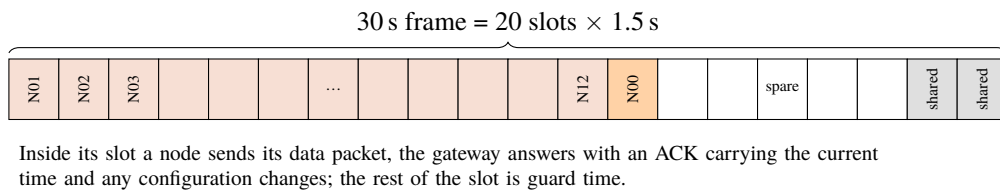


Figure 2.2: TDMA frame of Design B

Where the numbers come from. The frame is 30 s because that is the sampling period (same as Design A). 20 slots give 12 indoor nodes + N00 + 5 spare slots for extra nodes + 2 shared slots for urgent packets. Each slot is 1.5 s, while a data packet (about 67 ms) plus ACK (about 41 ms) needs roughly 0.1 s, so there is a large guard time. That guard time matters: a node's crystal clock drifts only a few milliseconds between two ACKs, and every ACK re-synchronises it, so nodes never slide into a neighbour's slot and no separate beacon is needed.

2.4.3 Reliable delivery

Radio packets can still be lost (interference, a person standing next to the antenna). Each packet is acknowledged; if no ACK comes back, the node retries up to three times, and if that fails the data stays in a **24-hour store-and-forward buffer** and is sent with the next packet. The gateway reorders by timestamp and drops duplicates using sequence numbers. Data may arrive late, but it is not lost.

2.4.4 Adaptive reporting: saving energy without delaying alerts

The node *samples* every 30 s (sensors cost very little energy), but *transmits* adaptively – because the radio is by far the largest energy consumer:

- **Sparse mode:** while values are far from any alert threshold, send one packet every 120 s containing the last four samples.
- **Dense mode:** when a value is close to an alert threshold (e.g. within 1 °C of the warning level) or changes quickly, send every sample.
- **Urgent:** a sensor fault is sent immediately in a shared slot.

The gateway pushes the “close to threshold” levels down to the nodes inside ACKs. Nodes never decide on alerts; they only decide *when to talk*. The simulation in Chapter 5 compares this policy with the only fixed schedule that has the same freshness – sending every sample (every 30 s): alert delay is the same, while the number of transmissions falls by more than half.

2.4.5 Energy

Each node runs on a LiFePO₄ 18650 cell (chosen because it tolerates greenhouse heat better than ordinary Li-ion) and a small solar panel. Between measurements the microcontroller sleeps, the radio is off and the light sensor is powered down. An important observation from our energy budget: once the radio is duty-cycled this aggressively, the node’s average consumption becomes *comparable to the battery’s own self-discharge*. Further radio optimisation therefore brings little; the right answer is a small solar panel, which easily covers the daily need even in the shade of the roof. The battery then only has to bridge cloudy periods.

2.4.6 Security

Unlike a cable, a radio channel can be overheard or spoofed – for example, an attacker could replay old “normal temperature” packets to hide a real problem. Each node has its own AES-128 key; packets are encrypted and carry a short integrity code, and the gateway rejects any packet whose frame counter does not increase (anti-replay).

2.5 Summary of the two infrastructures

Table 2.3: How each design meets the infrastructure requirements

Requirement	Design A (RS-485 ring)	Design B (LoRa star)
Completeness	Retry once per cycle; gateway reads node buffers after a restart	ACK, 3 retries, 24 h buffer
Timeliness	Fixed, under one cycle	Variable (up to the report period); near-threshold data sent immediately
Integrity, order	Modbus CRC; timestamps from synchronised broadcast	Radio CRC + integrity code; sequence numbers; clock sync via ACK
Single failures	One cable break: no node lost	Failures are independent per node
Energy	Not an issue (24 V over cable)	Sleep, SF7, adaptive reporting, solar
Security	Physical access needed	Encryption, integrity code, anti-replay
Main cost	About 130 m of cable and installation work	More complex firmware; batteries

Chapter 3

Criterion 2 – Data Correctness

3.1 Where errors come from

A value can go wrong anywhere along its path: *sensor* $\rightarrow$ *node* $\rightarrow$ *link* $\rightarrow$ *gateway*. No single technique catches every error, so we use three layers, each aimed at a different kind of error (Table 3.1). The first two layers are identical in A and B; the third differs only in how transmission errors are handled.

Table 3.1: Three layers of data correctness

Layer	Error type	Countermeasure	A vs B
Sensor	Systematic offset, sun heating the sensor, condensation, poor placement	Good sensors, radiation shield, 2-point calibration, heater	Same
Node	Random noise, electrical spikes, impossible values	Repeated reads, median, range and rate checks, quality flag	Same
Gateway	Drifting / stuck / dead sensors; lost, corrupted, late or forged packets	Neighbour comparison, zone median, sequence numbers, timestamps	A: CRC + retry. B: CRC + integrity code, ACK, buffer, <i>late-data handling</i>

3.2 Sensor layer: prevent errors at the source

- **Radiation shield.** Direct sun on a bare temperature sensor makes it read several degrees above the real air temperature – the largest single error source in a greenhouse. A ventilated multi-plate shield removes it.
- **Placement.** At canopy height, away from misting nozzles, fans and glass walls; light sensor on top of the mast, facing up.
- **Two-point calibration.** Before installation each sensor is compared with a reference at two temperatures; the correction $x_{true} = a x_{measured} + b$ is stored in the node and repeated periodically.
- **Condensation.** After misting, water on the humidity sensor makes it stick near 100 %RH. The

SHT40's built-in heater is switched on briefly to dry it; readings during recovery are flagged SUSPECT.

- **Light saturation.** The TSL2591 tops out at about 88 klux. Outdoor sun exceeds this, so N00 looks through a diffuser of known attenuation. Indoor midday light can also come close to the limit (roof transmission is often 60–80 % of outdoor light), so indoor nodes reduce gain and integration time automatically, and a reading at full scale is flagged SUSPECT; if this happens often, a diffuser is added to the indoor sensors too.

3.3 Node layer: clean every sample

Every cycle, each node runs the procedure of Figure 3.1 and attaches a quality flag to the value.

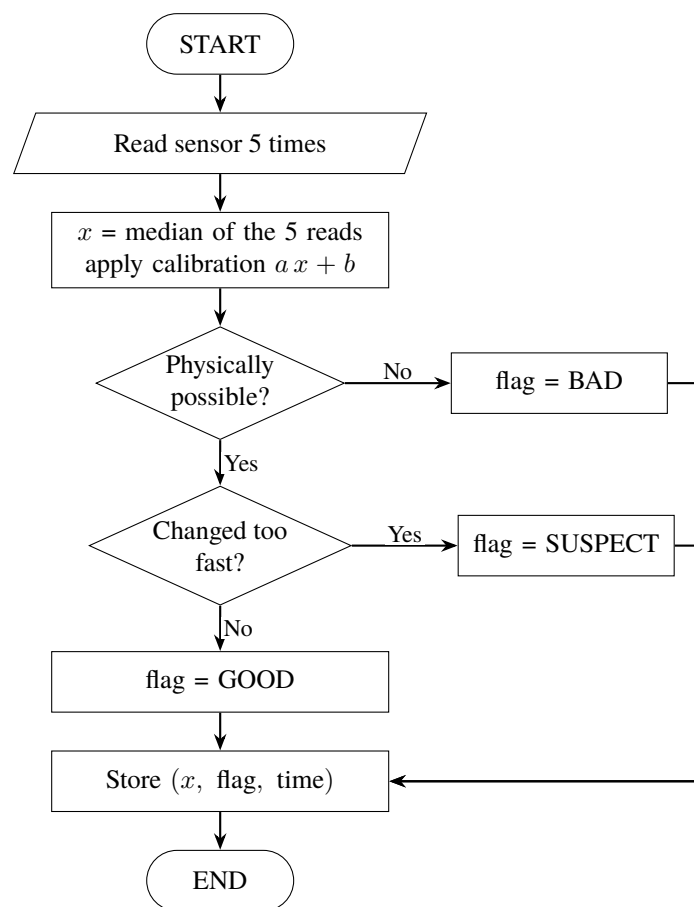


Figure 3.1: Filtering and validation on every node (both designs)

Where the numbers come from. *Five reads, median:* five reads take only about 50 ms. The median of five ignores up to two corrupted reads, whereas an average would be dragged by a single spike. *Physically possible:* temperature 0–60 °C, humidity 0–100 %, light not negative – anything else is a hardware fault. *Too fast:* a change of more than 3 °C per minute. Real greenhouse air has too much thermal inertia to change that fast, so such a jump is almost certainly a glitch; it is flagged, not deleted, in case it is real.

Only **GOOD** values may trigger or clear environmental alerts. **SUSPECT** values are still displayed (marked), **BAD** values are not.

We deliberately do *not* put a smoothing filter (moving average) on the alert path: it would delay the alert by about a minute, and the alert logic already rejects short disturbances on its own (Chapter 4). Smoothing is applied only to dashboard curves.

3.4 Gateway layer: catch what a single node cannot see

A node cannot know that it is drifting – its readings look perfectly plausible. Only a comparison with other nodes reveals it. This is why fault detection runs on the gateway.

3.4.1 Detecting faulty sensors

- **Drift.** For each node, compute the difference between its value and the median of its grid neighbours. If the difference stays above 2 °C for 30 minutes while the neighbours agree with each other, the node is marked as drifting and removed from calculations; maintenance is notified.
- **Stuck value.** A node whose reading has not changed at all for 30 minutes while all other nodes are changing is marked BAD.
- **Dead node.** No data for three expected deliveries: three polls in A (1.5 min), three report periods in B (6 min). B detects a dead node more slowly – one of its costs.

Where the numbers come from. 2 °C: normal differences between neighbouring nodes (sunny side vs. shaded side) are around 1 °C, and each sensor is accurate to about ± 0.2 °C, so 2 °C is clearly abnormal without triggering on normal gradients. 30 minutes: a local event (a door opened, a heater near one node) passes within minutes; a drift does not. Requiring persistence avoids blaming a healthy node for a real local event.

3.4.2 Zone value: tolerate one bad node per zone

Alerts are never based on a single node. For each zone we compute the median of its GOOD nodes and classify the result by how many GOOD nodes there are:

Table 3.2: Zone validity

GOOD nodes	Zone status	Zone value and use
4 or 3	VALID	Median of the GOOD nodes; used for alerts. One wrong value cannot move a median of 3 or 4 values.
2	DEGRADED	The “median” of two values is just their average, so one bad node <i>could</i> pull it. Used for alerts only if the two nodes agree within 1 °C; otherwise treated as invalid. A device alert “zone degraded” is raised in both cases.
0 or 1	INVALID	No zone value; the zone’s alert FSMs go to SENSOR_FAULT.

So, as long as a zone is **VALID**, even a faulty node that has not yet been detected cannot cause a **false alarm**. In the **DEGRADED** state protection is weaker, which is why it requires agreement and is reported for maintenance. An **INVALID** zone is *never* silently treated as normal.

Where the numbers come from. 1 °C: two healthy nodes in the same zone normally differ by less than about 1 °C (sunny vs. shaded side), so a larger disagreement means one of them cannot be trusted and we cannot tell which.

This argument assumes nodes fail independently. That holds for B (separate radio links) but not for a plain bus in A, where one cable break takes out many nodes together – which is exactly why Design A uses the ring bus.

3.4.3 Real event or faulty sensor?

Table 3.3: Telling a real event from a sensor fault

Clue	Real event	Sensor fault
Space	Several neighbouring nodes move together	One node deviates alone
Speed	Consistent with thermal inertia (minutes)	Sudden jump, or no change at all
Cause	Explained by sun, outdoor node N00, a stopped fan	No link with sun, N00 or other nodes
Time	Follows day/night, recovers when the cause ends	Persists, never recovers

The hard case is a *real* hot spot affecting only one node (e.g. a fan next to it failed). The zone median ignores it, but the separate “uneven temperature” alert (Section 4.6) still catches it.

3.4.4 Missing data vs. late data

This is where A and B differ most. In A, a value not received in its polling cycle really is missing. In B, a value may simply not have been *sent yet* (sparse mode sends every 120 s). If the gateway treated “not yet arrived” as “missing”, it would constantly declare sensor faults. We therefore use one extra quality level:

Table 3.4: Quality levels and how they are used

Level	Meaning	Used for alerts
GOOD	Passed all checks	Yes
SUSPECT	Changed too fast, or node suspected of drift	No
BAD	Impossible value, stuck, sensor error	No
PENDING (B only)	Not received yet, but still within the expected delivery delay	No, and <i>not penalised</i>
MISSING	Not received after the expected delivery delay	No

Two further rules: missing values are **never interpolated for alerts** (an estimate is not a measurement; interpolation is used only to draw heat maps, labelled “estimated”), and every data gap is logged with its cause (packet loss, node fault, cable fault).

3.5 Error scenarios at a glance

Table 3.5: Main error scenarios and how they are handled

	Scenario	Detected by	Handling
A+B	Electrical spike	Median, rate check	Spike discarded
A+B	Sun heating the sensor	Deviation from neighbours grows with light	Radiation shield; inspect shield
A+B	Condensation	RH stuck near 100 %	Heater, re-read, SUSPECT meanwhile
A+B	Slow drift	Neighbour comparison	Removed from zone value; maintenance
A+B	Stuck sensor	No variation for 30 min	BAD; device alert
A+B	Node crash	Watchdog; no data	Auto-reset; dead-node alert
A+B	Corrupted packet	CRC	Discarded; A re-polls, B retransmits
A	Cable break	P2 loses continuity	Split mode, no node lost; break located
B	Radio loss (wet leaves, people)	Missing sequence numbers	ACK, retry, 24 h buffer
B	Forged/replayed packet	Integrity code, frame counter	Rejected; security alert
B	Low battery	Voltage reported in every packet	Early warning; report less often
A+B	Gateway down	Hardware watchdog; external heartbeat	UPS, auto-restart, relay-driven siren; nodes buffer data

Chapter 4

Criterion 3 – Data Analysis for Alerts

4.1 The two ways an alert system fails

- **False alarm:** an alert when nothing is wrong. Frequent false alarms teach the grower to ignore the system – including the next real alarm (*alarm fatigue*).
- **Missed or late alarm:** a real problem is not reported in time, and crops are damaged.

These pull in opposite directions: filtering harder removes false alarms but delays real ones. Our design uses several *independent* filters against false alarms, and several *counter-measures* that keep real alarms fast. The analysis logic is **identical in A and B**; only one parameter (the expected delivery delay) differs.

4.2 Processing pipeline

Every 30 s the gateway runs the steps of Figure 4.1.

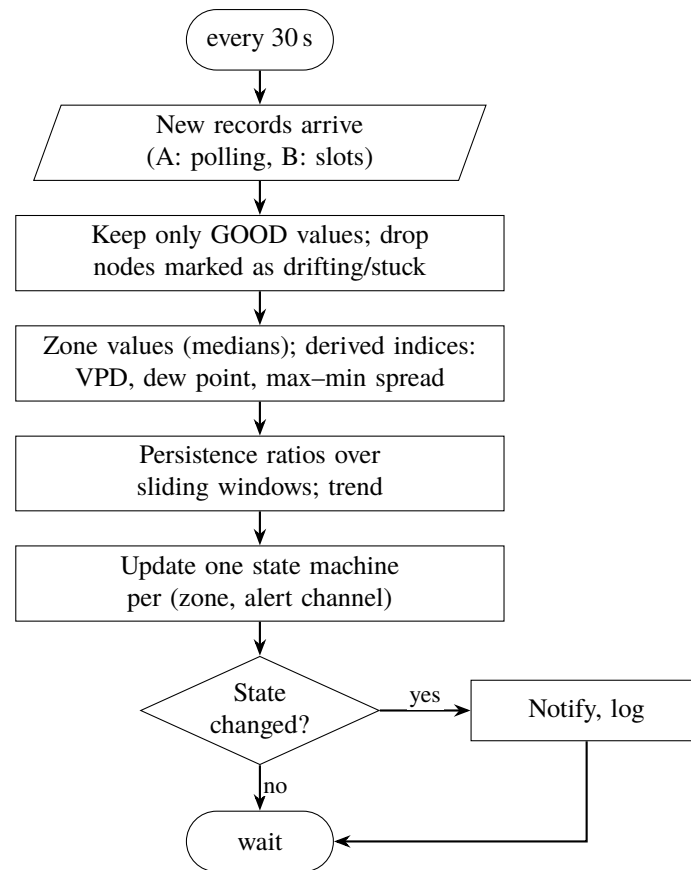


Figure 4.1: Alert analysis pipeline on the gateway

4.3 The alert state machine

Each pair (zone, alert channel) – for example “zone Z2, high temperature” – has its own finite state machine (FSM). All FSMs follow the same template (Figure 4.2); only thresholds change. Three design choices:

- **Moore outputs plus entry actions:** the lights and the siren are Moore outputs – they depend only on the current state, so they do not flicker when data is noisy. Messages are *entry actions*: sent once when a state is entered, plus a periodic reminder while in ALARM.UNACK.
- **Hierarchical:** NORMAL, WARNING and ALARM sit inside a MONITORING super-state. The transition to SENSOR_FAULT is defined once, at the super-state level. ALARM has two sub-states: UNACK (siren on) and ACK (operator acknowledged; siren off, red light stays).
- **Clean inputs only:** the FSM sees zone values built from GOOD data, never a raw node reading.

Where the numbers come from. The 32 °C / 35 °C levels are typical reference values for common vegetables and are *configurable per crop*; the method does not depend on them. The exit levels (30 °C, 33 °C) are 2 °C below the entry levels – see hysteresis below.

Execution rules.

- Each FSM moves at most one step per cycle. If several conditions hold, a fixed priority decides: sensor fault (T5) > escalation (T7, T3) > T1 > de-escalation (T4, T2). The machine is therefore

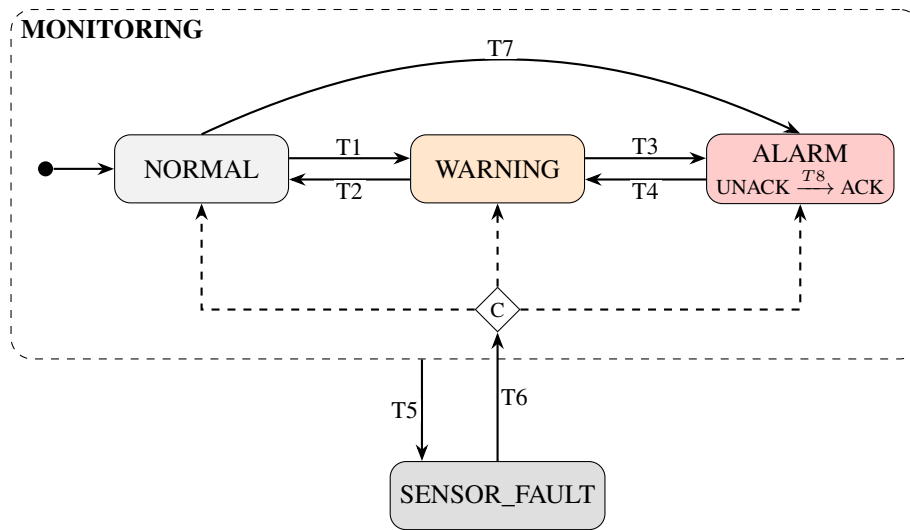


Figure 4.2: Hierarchical alert FSM (one zone, high-temperature channel); C is the choice point used when recovering from a sensor fault

Table 4.1: Transitions of the high-temperature FSM

ID	Transition	Condition (“sustained” is defined in Section 4.4)
T1	NORMAL → WARNING	$T > 32\text{ °C}$ sustained over 5 min
T2	WARNING → NORMAL	$T < 30\text{ °C}$ sustained over 5 min
T3	WARNING → ALARM	$T > 35\text{ °C}$ sustained over 3 min, <i>or</i> rising trend (Section 4.5)
T4	ALARM → WARNING	$T < 33\text{ °C}$ sustained over 5 min
T7	NORMAL → ALARM	$T > 35\text{ °C}$ sustained over 3 min (sudden jump, no need to pass WARNING)
T8	ALARM.UNACK → ALARM.ACK	Operator acknowledges
T5	MONITORING → SENSOR_FAULT	Zone INVALID, or DEGRADED with its two nodes disagreeing (Table 3.2), or too few recent samples
T6	SENSOR_FAULT → C	Reliable data is back; C re-enters ALARM, WARNING or NORMAL according to current data. A return to ALARM always enters ALARM.UNACK (siren on): an acknowledgement given before the fault may no longer apply

deterministic.

- **Escalate fast, de-escalate carefully:** there is a shortcut `NORMAL` → `ALARM` but no shortcut `ALARM` → `NORMAL`. Missing a danger is worse than clearing an alarm late.
- **Fail-safe:** missing data never leads to `NORMAL`; it leads to `SENSOR_FAULT`, which is reported clearly.
- **Messages as entry actions**, with a reminder every 15 minutes while an alarm is unacknowledged – no message flood.

4.4 Four mechanisms against false alarms

4.4.1 Persistence over a sliding window

A condition (e.g. “ $T > 32\text{ °C}$ ”) counts as **sustained** when

$$\frac{\text{\# recent samples satisfying the condition}}{\text{\# recent samples available}} \geq 0.8, \quad (4.1)$$

with at least 60 % of the expected samples available. The window is 5 min (10 samples) for `WARNING` and 3 min (6 samples) for `ALARM`.

Where the numbers come from. *80 % rather than 100 %:* one noisy or missing sample does not reset the count, so a real event is not delayed by a single glitch. *At least 60 % available:* a decision made on too few samples is not trustworthy; below that, the zone goes to `SENSOR_FAULT` instead. *5 and 3 minutes:* misting or an opened door disturbs the air for a minute or two, while crop damage from heat needs longer exposure – so a few minutes of confirmation removes most false alarms at little cost. `ALARM` uses the shorter window because it is the more dangerous level.

4.4.2 Hysteresis – avoiding a “Zeno” alarm

With a single threshold, a noisy value hovering around 32 °C would switch the alarm on and off over and over – in the ideal continuous model, infinitely often in finite time (a Zeno-like effect). Two things prevent this. First, the exit threshold is 2 °C below the entry threshold, far larger than the noise of a zone value, so every change of state needs a real temperature swing of more than 2 °C . Second, every transition needs a minimum amount of evidence, which bounds how fast states can change:

- de-escalation needs 8 of 10 samples beyond the exit level: at least 4 min;
- escalation by threshold needs 5 of 6 samples above the `ALARM` level: at least 2.5 min; escalation by trend needs the trend to hold for 3 cycles: at least 1.5 min.

The fastest possible oscillation is `WARNING` → `ALARM` → `WARNING`, which takes at least $1.5 + 4 = 5.5$ min for two changes, i.e. **at most about 22 changes per hour per channel** (about 18 if the trend rule is not involved). This is a finite bound, so there is no Zeno behaviour; in the simulation the real number was far lower (Chapter 5).

4.4.3 Zone median

As shown in Section 3.4.2, while a zone is VALID (≥ 3 GOOD nodes) one wrong node cannot move the zone value, so a single faulty sensor cannot raise an alarm. With only 2 GOOD nodes the zone value is used only if they agree within 1 °C.

4.4.4 Late data is not missing data (watermark)

The persistence ratio uses *the most recent samples that have actually arrived*, looking back over the window plus the design’s expected delivery delay (the *watermark*): one cycle for A, one report period plus slot waiting for B. Early data (always in A; in B’s dense mode) is used immediately, so alerts are not slowed down. Data that is simply not due yet (PENDING) is not counted as missing, so B does not fall into false sensor faults.

Where the numbers come from. Without this rule, a version of the simulation of Design B (120 s report period) with a plain clock-aligned window produced about five times as many state changes as Design A – roughly 180 instead of 36 per week, mostly false SENSOR_FAULT entries – the gateway kept mistaking “not sent yet” for “lost”. This is a concrete example of the communication design and the analysis design interacting, found only by evaluation (Chapter 5).

4.5 Keeping real alarms fast

- **Trend alert.** If the zone temperature has risen by more than 2 °C in 10 minutes – computed as the median of the last 3 minutes minus the median of 3 minutes taken 10 minutes earlier, and holding for 3 consecutive cycles – WARNING escalates to ALARM *before* 35 °C is reached. Medians and the 3-cycle requirement stop a short misting dip from faking a trend.
- **Shortcut** NORMAL → ALARM for sudden jumps.
- **Shorter window** for ALARM than for WARNING.
- **The gateway watches itself:** a hardware watchdog relay sounds the siren if the gateway software stops, and a heartbeat to the outside reports a dead gateway.

Where the numbers come from. 2 °C in 10 minutes: the normal morning warm-up is a fraction of a degree per 10 minutes, whereas a failed fan heats the air several times faster. 2 °C separates the two clearly.

4.6 Alerts from derived indices and from space

Raw thresholds miss some risks, so the gateway also computes:

- **Vapour pressure deficit (VPD)** – how strongly the air “pulls” water from leaves:

$$e_s = 0.6108 \exp\left(\frac{17.27 T}{T + 237.3}\right) \text{ kPa}, \quad VPD = e_s \left(1 - \frac{RH}{100}\right). \quad (4.2)$$

Too low: fungal disease risk; too high: plant water stress. Neither is visible from temperature or humidity alone.

- **Dew-point margin** $T - T_d$ (Magnus formula): when it is small, water condenses on leaves, typically at night.
- **Daily light integral (DLI)**: plants need a total amount of photosynthetic light per day, not a momentary value. The TSL2591 measures lux, so we convert with the usual factor for *sunlight*, $PPFD \approx E_{lux}/54 \mu\text{mol m}^{-2} \text{s}^{-1}$ (1 klux $\approx 18.5 \mu\text{mol m}^{-2} \text{s}^{-1}$), and sum over the day:

$$DLI = \sum_k PPFD_k \cdot \Delta t_k \cdot 10^{-6} \text{ mol m}^{-2} \text{ day}^{-1}, \quad (4.3)$$

checked once in the evening. The factor depends on the light spectrum: it is only approximate under shading nets and wrong for LED grow lights, where a lamp-specific factor or a PAR (quantum) sensor is needed.

- **Uneven temperature**: the spread between the hottest and coldest GOOD nodes. A large spread usually means a fan or vent in one area has failed – an alert that only a multi-point system can give.

All use the same FSM template (WARNING level only), with windows adapted to how slowly each quantity changes.

4.7 Device alerts are separate

Environmental alerts (“the greenhouse is too hot”) and device alerts (“a sensor is broken”) go to separate channels, so the grower knows whether to act on the greenhouse or call maintenance:

- **Both designs**: drifting, stuck or dead sensor; zone in SENSOR_FAULT; gateway heartbeat lost.
- **Design A only**: cable break with the broken segment; rising CRC errors (moisture in a connector).
- **Design B only**: low battery; decreasing link margin; invalid integrity code (possible attack); node missing its slot.

Chapter 5

Performance Evaluation

The three mechanisms above were evaluated with the standard 10-step performance-evaluation procedure. Table 5.1 summarises the steps; the metrics are chosen *separately* for each design because each design can fail in different ways.

5.1 The 10 steps in brief

Table 5.1: The 10-step evaluation applied to A and B

Step	Our choice
1. Goals, system	Compare A and B fairly (stated neutrally, not “prove B is better”); tune each design’s parameters. System = nodes + network + gateway. The phone network and the Internet are outside: their delay does not depend on A or B.
2. Services, outcomes	Data collection, alerting, fault diagnosis. Each can succeed, go wrong (wrong/late value, false alarm), or fail (lost data, missed alarm).
3. Metrics	One metric per outcome (Table 5.2); separate sets for A and B.
4. Parameters	System: window lengths, thresholds, bus speed (A), report period and radio settings (B). Workload: daily temperature cycle, fan failures, misting, sensor noise and faults, packet loss.
5. Factors	Design (A/B); trend alert on/off; B: report period, dense mode on/off, packet-loss rate; plus a naive single-threshold baseline.
6. Technique	Analytical models first (bus timing, link budget, collisions, energy), then simulation for time-dependent behaviour, then measurement on a prototype (planned). Each technique cross-checks another.
7. Workload	7 simulated days: daily cycle close to the warning level, 2 fan failures per week, misting 3 times a day, sensor spikes, one drifting and one stuck sensor, random packet loss.
8. Experiments	12 configurations $\times$ 10 repetitions. Repetition r uses the same random weather and faults for every configuration, so differences come only from the design.
9. Analysis	Mean with 95 % confidence interval, plus the worst case (for alarms, the worst case matters most).
10. Presentation	Tables and a chart (below).

5.2 Metrics chosen for each design

Table 5.2: Metrics per design and why

	Metrics	Why this set
Both designs	Alert delay (mean and worst); false alarms; missed events; number of state changes; measurement error; fault-detection time; data completeness	These are the outcomes of the three services; they are used to compare A with B
Design A only	Poll cycle time and bus load; nodes still reachable after a cable break; voltage at the last node	A's weak point is correlated failure (a cable break), not energy. Data age is excluded: it is constant in A
Design B only	Link-layer delivery ratio; collision probability; transmissions per node per day; average current / autonomy; link margin; fairness of delivery across nodes (Jain index)	B's weak points are the shared radio channel and the battery. Link margin is "nominal-is-best": too low loses packets, too high wastes energy. The fairness index reveals a single far node that keeps losing packets, which an average would hide

5.3 Key results

The results below come from our simulator (Appendix A). " $x \pm y$ " means mean $\pm$ half-width of the 95 % confidence interval over the repetitions.

Table 5.3: Key simulation results

Question	Result	Meaning
Does node filtering work? (criterion 2)	Error of one reading 0.67°C → after median 0.08°C	Occasional spikes dominate the raw error; the median removes them
Are faulty sensors caught? (criterion 2)	Stuck: after 30 min. Drift: caught at about 2.3°C offset	Matches the 30 min / 2°C rules; caught before the error becomes harmful
Does the alert logic prevent flip-flopping? (criterion 3)	State changes per week, 3 zones: naive threshold 576 ± 79 , our design 35 ± 6	The remaining changes are genuine warm afternoons crossing 32°C
Are real events caught? (criterion 3)	20 of 20 fan failures detected, 0 false ALARMS, in every configuration	The false-alarm filters do not hide real events

5.3.1 Does the network change the alert delay? (paired comparison)

Because every configuration was run on the *same* 20 fan failures (common random numbers), the fair way to compare two designs is per event: compute "delay of B – delay of A" for each of the 20 failures, then a confidence interval on that difference. This removes the event-to-event variation that both designs share, and is much more precise than comparing two separate intervals.

Where the numbers come from. *Alert delay* (minutes) is measured from the moment the true zone temperature crosses 35°C to the moment the FSM enters ALARM; it excludes SMS delivery. About 2.5 min of it is intentional: the 3-minute ALARM window must see 5 of 6 samples above 35°C . So A's 2.7 min is almost entirely the design's own confirmation time, not network delay – which is why B can match it. A *negative* delay means the trend rule raised the alarm before 35°C was reached. The trend rule looks 10 minutes back, and B's older samples are still sparse at that point, so B's trend alarm comes about 50 s later than A's.

Table 5.4: Alert delay and paired differences (20 fan failures; trend alert off unless stated)

Configuration	Mean delay	Worst	Paired vs. A (95 % CI)	Reading
A (RS-485)	2.70 ± 0.37	6.0	–	Reference
B, fixed 30 s	2.75 ± 0.43	6.5	+0.05 [–0.02, +0.12]	No significant difference
B, adaptive (120 s + dense)	2.75 ± 0.43	6.5	+0.05 [–0.02, +0.12]	No significant difference; at most ≈ 7 s slower
B, fixed 120 s	3.40 ± 0.53	7.5	+0.70 [+0.43, +0.97]	Significantly slower; worst case near the 8 min limit
A, trend on	-1.23 ± 0.74	1.5	–	Alarm before 35 °C is reached
B adaptive, trend on	-0.38 ± 0.80	3.0	+0.85 [+0.61, +1.09]	Significantly later than A, but still before 35 °C on average

5.3.2 Does adaptive reporting save energy? (criterion 1)

The fair baseline for adaptive reporting is the fixed schedule that gives the *same alert delay*: sending every sample, every 30 s. Table 5.4 shows that their delays are identical for every one of the 20 failures (paired difference exactly 0).

Table 5.5: Transmissions per node per day (5 % packet loss, retries included)

Reporting policy	Transmissions/node/day	Mean delay (trend off)
Fixed 30 s (same-delay baseline)	3,031	2.75 min
Adaptive: 120 s + dense near thresholds	1,280 (–58 %)	2.75 min
Fixed 120 s	758	3.40 min

Where the numbers come from. 3,031: one sample every 30 s is 2,880 packets a day, plus about 5 % retransmissions. 1,280: sparse packets every 120 s (720 a day) plus dense packets during the warm hours when values are within 1 °C of the warning level. Since the radio dominates the node’s energy, 58 % fewer packets means roughly half the radio energy for the same alert delay. Fixed 120 s is cheaper still, but significantly slower (Table 5.4).

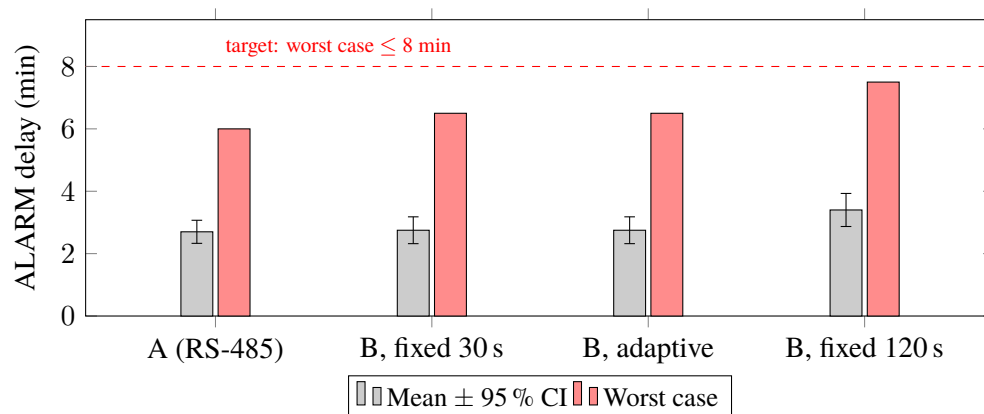


Figure 5.1: Alert delay with the trend alert switched off (isolates the effect of the network)

5.4 Evaluation changed the design

The evaluation was not only a final check – it fixed three design flaws, all caused by the interaction between communication and alert analysis:

1. B with sparse reporting and a clock-aligned window caused false `SENSOR_FAULT` states (about 180 instead of 36 state changes per week) $\Rightarrow$ the watermark and the `PENDING` quality level were added.
2. The first watermark version made B slower than A $\Rightarrow$ it was changed to use “the most recent samples that have arrived”.
3. Sending a single early packet when nearing a threshold did not reduce delay, because persistence needs several fresh samples $\Rightarrow$ replaced by dense mode.

The simulation covers only the high-temperature channel with independent packet losses. A prototype (one zone per design, two weeks, reference thermometer, current meter, cable-cut and gateway-power-cut tests) is planned to confirm these results. [Add measurement results if a prototype is built.]

Chapter 6

Comparison and Recommendation

Table 6.1: Design A vs. Design B along the three criteria

	A: RS-485 ring bus	B: LoRa star, TDMA, adaptive
<i>Communication infrastructure</i>		
Delivery	Deterministic, fixed delay	Variable delay; relies on ACK, retries, buffering
Failure behaviour	Survives one cable break; two breaks isolate a segment	Independent per node; sensitive to wet foliage and interference
Power	From the cable	Battery + solar
Installation	About 130 m of cable in a wet environment; hard to rearrange	Fast; nodes can move with the crop layout
Security	Physical access needed	Needs encryption and key management
<i>Data correctness</i>		
Sensor and node layers	Identical	
Transmission errors	CRC + re-poll	CRC + integrity code + retransmission
Dead-node detection	Faster (3 polls)	Slower (3 report periods)
<i>Data analysis for alerts</i>		
Alert logic	Identical; only the watermark differs	
Alert quality	Same false-alarm and detection results. Delay with trend alert off: no significant difference (paired B – A: +0.05 min, 95 % CI [–0.02, +0.12]). With trend alert on: B about 0.85 min later, still before 35 °C on average	
Software complexity	Low	Higher (slots, sync, energy, buffer, crypto)

Main finding. Because the alert logic absorbs most of the difference in delivery delay, the simulation shows *no significant difference* in threshold-based alert delay, and identical false-alarm and detection results. The only measurable penalty of B is that its early trend alarm comes slightly later. The choice is therefore decided mainly by installation, flexibility, maintenance and failure behaviour.

- **Choose A** for a permanent greenhouse with a high-value crop, available wiring labour, and a preference for the simplest, most deterministic system.
- **Choose B** when the layout changes from season to season, the greenhouse is rented or temporary,

fast deployment is needed, or cabling in wet conditions is a maintenance concern.

- **Our recommendation for this assignment:** in a 20×50 m greenhouse the cable for A is short (about 130 m, along the two node columns), so A's installation cost is modest. However, a vegetable greenhouse in Vietnam typically changes its layout between crops and is very humid, where cable connectors are a frequent point of failure. We recommend **Design B with adaptive reporting**, on condition that the link margin is verified on site early in the morning (wet leaves). If that test shows long, correlated packet losses, Design A should be used instead.

[Confirm the final choice after prototype measurements.]

Chapter 7

Conclusion

- **Communication infrastructure:** Design A uses a ring-routed RS-485 bus that survives a cable break; Design B uses a LoRa star with a verified link margin, TDMA slots, acknowledged delivery with a 24-hour buffer, and adaptive reporting powered by a small solar panel.
- **Data correctness:** three layers – sensor protection and calibration, median filtering with quality flags on the node, and neighbour-based fault detection with zone medians on the gateway – so that a single faulty sensor can neither corrupt a zone value nor raise an alarm, and missing data is never mistaken for normal.
- **Data analysis for alerts:** a hierarchical Moore state machine with sliding-window persistence, hysteresis, trend detection, derived indices and a separate device-fault channel; a watermark lets the same logic run on both designs.
- **Evaluation** with the 10-step method, using paired comparisons on common random numbers, found no significant difference in threshold-based alert delay between A and B (B's trend alarm is slightly later), showed that adaptive reporting halves B's transmissions for the same delay as 30 s reporting, and improved the design three times. The six challenge families, with dependability first and energy promoted to second place for B, guided every trade-off.

Future work: a hybrid network (wired backbone + wireless mobile nodes), anomaly detection with machine learning on the gateway, and closed-loop control of fans, misting and shading.

Bibliography

- [1] P. Marwedel, *Embedded System Design*, 4th ed., Springer, 2021.
- [2] R. Jain, *The Art of Computer Systems Performance Analysis*, Wiley, 1991.
- [3] CO3053 lecture notes (Ch. 1 Introduction; L2 Platform Architecture; L3 Development Process; L4 State Machines & Flowcharts), HCMUT.
- [4] Tran Van Hoai, *Steps in Performance Evaluation and Selection of Techniques and Metrics*, lecture notes, HCMUT.
- [5] R. G. Allen et al., *Crop Evapotranspiration*, FAO Irrigation and Drainage Paper 56, 1998.
- [6] Sensirion SHT4x, ams-OSRAM TSL2591 and Semtech SX1276 datasheets; Modbus over Serial Line specification.

Appendix A

Simulator

The file `greenhouse_sim.py`, submitted with this report, implements the evaluation. It generates 7 days of true temperature for 12 nodes on a 2×6 grid in a 20×50 m greenhouse (daily cycle, spatial gradient, fan failures, misting), simulates sensor readings with noise, spikes, one drifting and one stuck sensor, models delivery for A (polling) and B (TDMA slots, adaptive reporting, losses, retries, buffer), and runs *the same* gateway code (fault detection, zone medians, alert FSM with watermark) for both designs.

```
pip install numpy
python3 greenhouse_sim.py 10 # 10 repetitions of every configuration
```

Limitations: only the high-temperature channel is simulated; packet losses are independent; the radiation-shield time constant and SMS delay are not included.